NitroStack Studio — Hackathon Handbook

Set up, build, and ship MCP apps with NitroStack Studio & NitroCloud

> **How to read this handbook:** Each task is broken into short numbered steps, with a screenshot beneath the relevant step.

Table of Contents

1. What is MCP?

MCP (Model Context Protocol) is an open standard that helps AI connect to different apps, tools, and data using one common way. Think of it like a **USB for AI** — just as a USB lets you connect many different devices without needing a different cable each time, MCP lets AI connect to different systems without creating a new integration for each one. This makes it easy, fast, and simple for AI to use different tools and perform tasks.

NitroStack Studio is the desktop IDE for building and testing MCP servers. **NitroCloud** hosts and deploys them so you can ship to ChatGPT and other clients.

2. Env Check

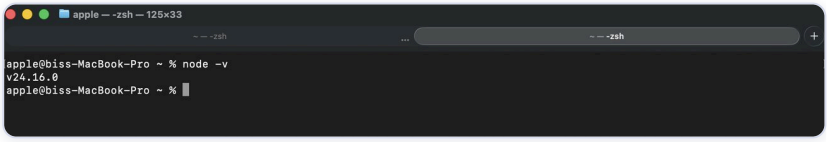
Before running Studio or creating a project, confirm your machine has the required tooling.

Tool	Version	Check command
Node.js	**20.x** recommended (18+ is the minimum Studio accepts)	`node -v`
npm	Any recent version (ships with Node)	`npm -v`
npx	Ships with npm	`npx -v`

Steps

1. Open a terminal and check Node.js. You should see something like `v20.x.x`; if it prints `v18` or newer, you're fine (v20.x is recommended).

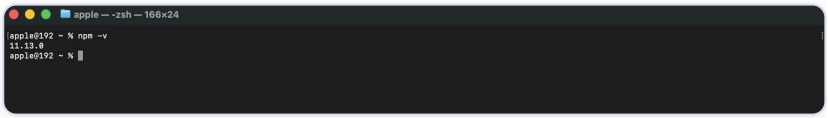
```
node -v
```



Terminal showing the installed Node.js version

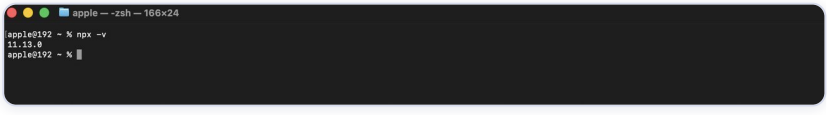
2. Check npm and npx are available.

```
npm -v
npx -v
```



Terminal showing npm and npx versions

3. If Node.js is missing or too old, install it from nodejs.org or let Studio handle it. When Studio detects a missing/old Node during onboarding, it shows a system check with **Install bundled Node.js** (installs v20.11.0), **Open nodejs.org**, and **Re-check** (re-runs the check after you install).



Studio environment / Node.js system check

> **Note:** Studio enforces **Node.js 18+** at minimum. Node 20 is used internally (bundled Node + cloud Docker images), so 20.x is the safest choice for the hackathon.

Your path — Env Check to shipped MCP

Once your environment is ready, this is the full handbook journey from **create** → **Studio** → **ship**:

Your path from Env Check to a live MCP in ChatGPT

Step	What you do	Handbook section
1	Confirm Node / npm / npx	Env Check (you are here)
2	Scaffold + code with CLI & SDK	CLI & SDK — Create a Project
3	Install the desktop app	Download & Install NitroStudio
4	Sign in to NitroCloud	Studio Login
5	Open / connect your project	Studio Project Connect
6	Test tools & vibe-code	Studio Features · Vibe Coding
7	Deploy to NitroCloud	Cloud & Deployment
8	Connect in ChatGPT	Connect to ChatGPT

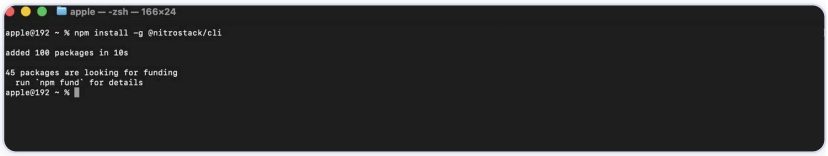
3. CLI & SDK — Create a Project

CLI scaffolds the app. **SDK** (`@nitrostack/core`) is what you write tools with. Flow: `init` → edit `@Tool` / `@Module` → `npm run dev` → connect in Studio.

CLI — scaffold

1. Install the CLI globally (or skip and use `npx`). Binary: `nitrostack-cli` .

```
npm install -g @nitrostack/cli
```



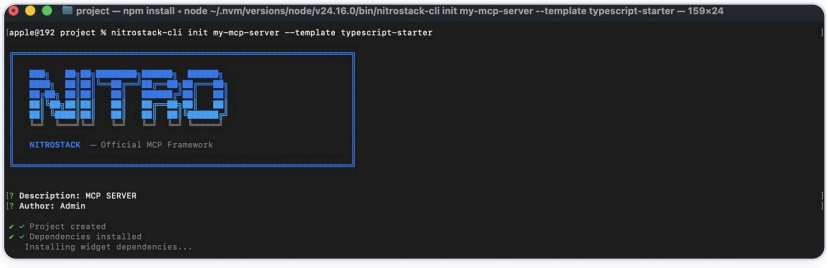
Installing the NitroStack CLI

2. Create a project:

```
nitrostack-cli init my-mcp-server --template typescript-starter
# or: npx @nitrostack/cli@latest init my-mcp-server --template typescript-starter
```

Template	When to use
`typescript-starter`	Default. Calculator + one widget. Best to learn.
`typescript-pizzaz`	Full demo: multiple widgets, tasks, follow-up tool calls.
`typescript-oauth`	OAuth 2.1 auth + protected resources.

Optional flags: `--description "<text>"` , `--author "<name>"` , `--skip-install` .



Scaffolding the project with npx

3. Run the MCP server + widgets (widget port defaults to **3001**):

```
cd my-mcp-server
npm run dev
```

Running npm run dev

Prod: `npm run build` then `npm run start:prod` .

Scaffold layout:

```
my-mcp-server/
├── src/
│   ├── index.ts             # McpApplicationFactory bootstrap
│   ├── app.module.ts        # @McpApp + root @Module
│   ├── modules/<name>/      # *.tools.ts / *.resources.ts / *.prompts.ts
│   ├── health/
│   └── widgets/             # React widgets (Next.js) for @Widget
├── package.json             # @nitrostack/core + @nitrostack/cli scripts
└── .env                     # NITRO_LOG_LEVEL, NITROSTACK_APP_MODE
```

> Studio needs `package.json`, `src/index.ts`, and `@nitrostack/core` to recognize a Nitro project.

SDK — write tools (required)

`@nitrostack/core` = decorator MCP framework (tools / resources / prompts + Zod). Deps already installed by `init`: `@nitrostack/core`, `zod`, `dotenv`.

1. Bootstrap (`src/index.ts`):

```
import 'dotenv/config';
import { McpApplicationFactory } from '@nitrostack/core';
import { AppModule } from './app.module.js';

const server = await McpApplicationFactory.create(AppModule);
await server.start();
```

2. Root app (`src/app.module.ts`):

```
import { McpApp, Module, ConfigModule } from '@nitrostack/core';
import { CalculatorModule } from './modules/calculator/calculator.module.js';

@McpApp({
  module: AppModule,
  server: { name: 'my-mcp-server', version: '1.0.0' },
  logging: { level: 'info' },
})
@Module({
  name: 'app',
  imports: [ConfigModule.forRoot(), CalculatorModule],
})
export class AppModule {}
```

3. Feature module — register controllers:

```
import { Module } from '@nitrostack/core';
import { CalculatorTools } from './calculator.tools.js';

@Module({
  name: 'calculator',
  controllers: [CalculatorTools], // add Resources / Prompts classes here too
})
export class CalculatorModule {}
```

4. Tool (what Studio / ChatGPT call):

```
import { ToolDecorator as Tool, Widget, ExecutionContext, z } from '@nitrostack/core';

export class CalculatorTools {
  @Tool({
    name: 'calculate',
    description: 'Perform basic arithmetic',
    inputSchema: z.object({
      operation: z.enum(['add', 'subtract', 'multiply', 'divide']),
      a: z.number(),
      b: z.number(),
    }),
  })
  @Widget('calculator-result') // optional UI route under src/widgets/
  async calculate(input: any, ctx: ExecutionContext) {
    ctx.logger.info('calculate', input); // use ctx.logger — not console.* (breaks STDIO)
    // …return { result, … }
  }
}
```

Same pattern for **resources** / **prompts**:

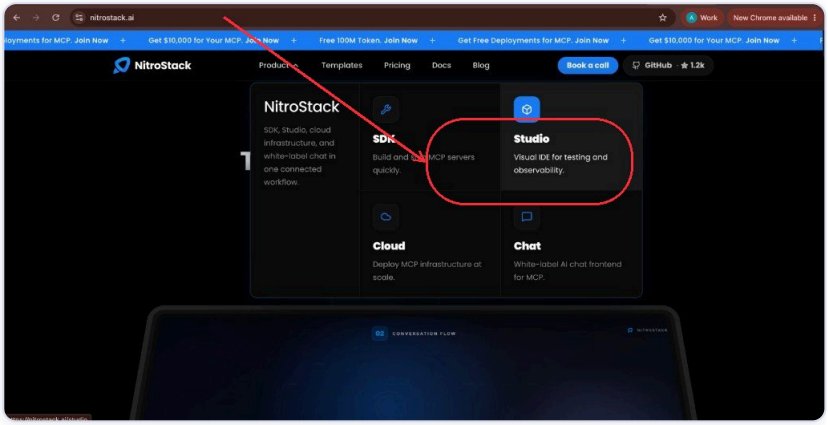
```
import { ResourceDecorator as Resource, PromptDecorator as Prompt } from '@nitrostack/core';

@Resource({ uri: 'app://config', name: 'Config', mimeType: 'application/json' })
async getConfig(uri: string, ctx: ExecutionContext) { /* … */ }

@Prompt({ name: 'help', description: 'Help prompt', arguments: [] })
async help(args: any, ctx: ExecutionContext) { /* return chat messages */ }
```

Need	Use
Scaffold / run	`@nitrostack/cli` → `init`, `npm run dev`
Write MCP logic	`@nitrostack/core` → `@McpApp`, `@Module`, `@Tool`, `@Resource`, `@Prompt`
Validate inputs	`zod` in `inputSchema`
Log safely	`ctx.logger` (never `console.log` on STDIO)

4. Download & Install NitroStudio

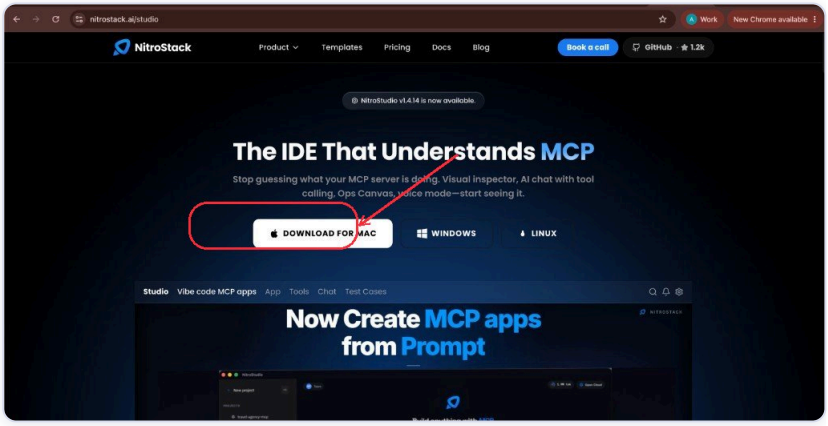
Install the **NitroStudio** desktop app before you sign in or connect a project.

1. Open nitrostack.ai in your browser. Click **Product** in the top nav, then click **Studio** (opens `https://nitrostack.ai/studio`).



nitrostack.ai — Product menu → Studio

2. On the Studio page, download the build for your OS — **DOWNLOAD FOR MAC**, **WINDOWS**, or **LINUX**.



NitroStudio download page — Mac / Windows / Linux

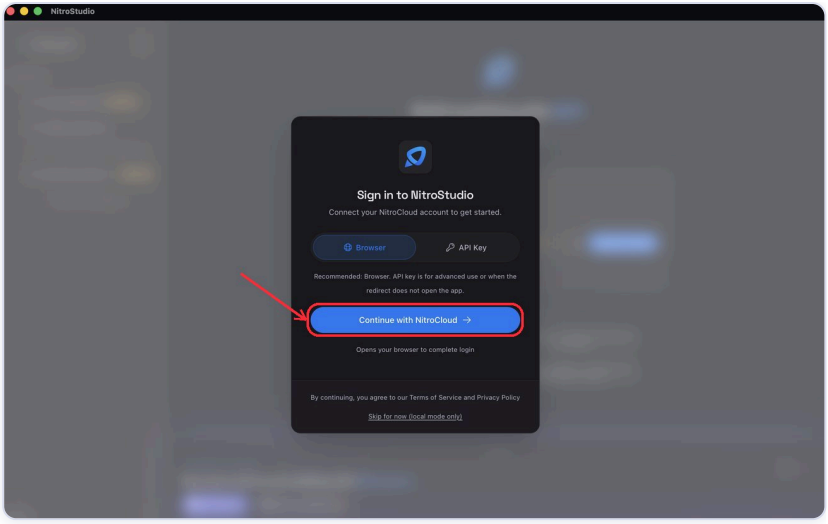
3. Open the downloaded installer and complete the install for your platform (macOS: open the `.dmg` / app; Windows: run the installer; Linux: follow the package / AppImage steps for your distro). Then launch **NitroStudio**.

5. Studio Login

Signing in connects Studio to your **NitroCloud** account, which unlocks AI Chat, Compose (Vibe Coding), and cloud deployment. Local features (Tools, Resources, Logs, Health) work without signing in.

Open the sign-in modal from any of these entry points:
- Launcher sidebar footer → **Sign in**
- App sidebar footer → **Connect to NitroCloud**
- Any gated feature (AI Chat / Compose) → **Sign In to NitroCloud**

The modal is titled **"Sign in to NitroStudio"** and has two tabs: **Browser** (recommended) and **API Key**.



The "Sign in to NitroStudio" modal with the Browser and API Key tabs

Method 1 — Browser (recommended)

This is the recommended, one-click flow. All of the steps below happen in and around the sign-in modal shown above.

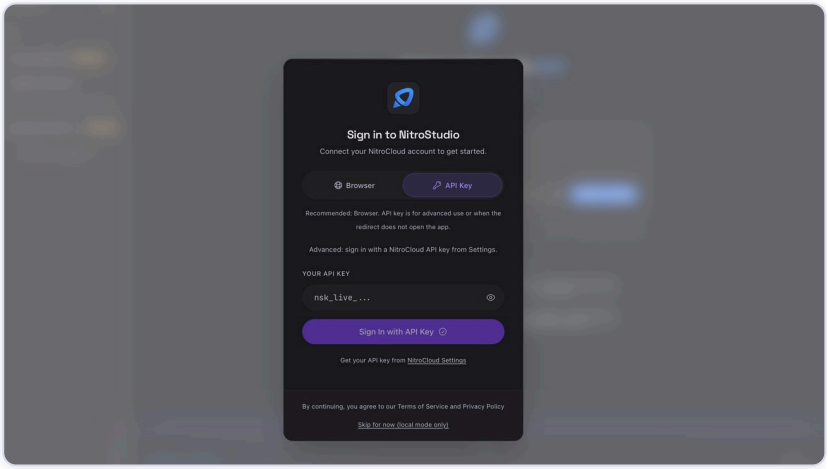
1. Make sure the **Browser** tab is selected (it's the default).
2. Click **Continue with NitroCloud**. Helper text under the button reads "Opens your browser to complete login".
3. Your system browser opens the NitroCloud login page. Complete sign-in there; Studio shows a **"Waiting for login..."** state while it waits.
4. Once done, Studio picks up the session automatically and shows a **"Welcome back, {your name}!"** toast. Your avatar/account menu now appears in the sidebar.

> **Tip:** If the browser finishes but Studio stays on "Waiting for login...", use **Switch to API Key** (appears after ~10 seconds) and follow Method 2.

Method 2 — API Key

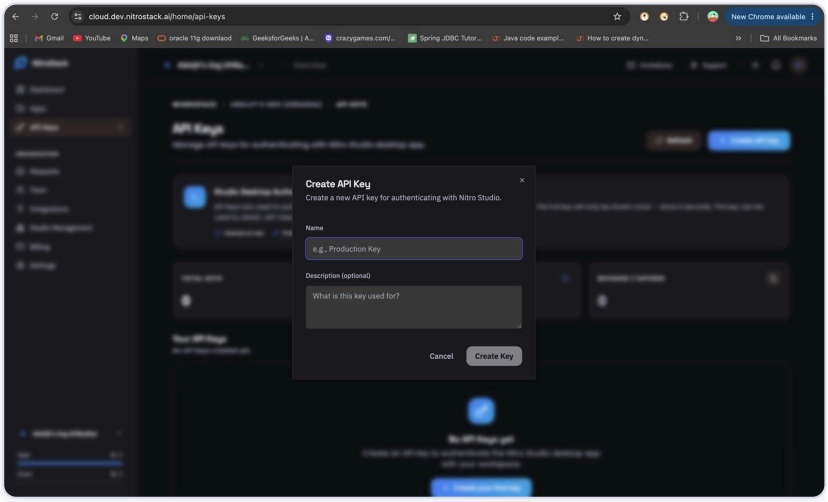
Use this when the browser redirect doesn't open the app, or for advanced/headless use.

1. Click the **API Key** tab.



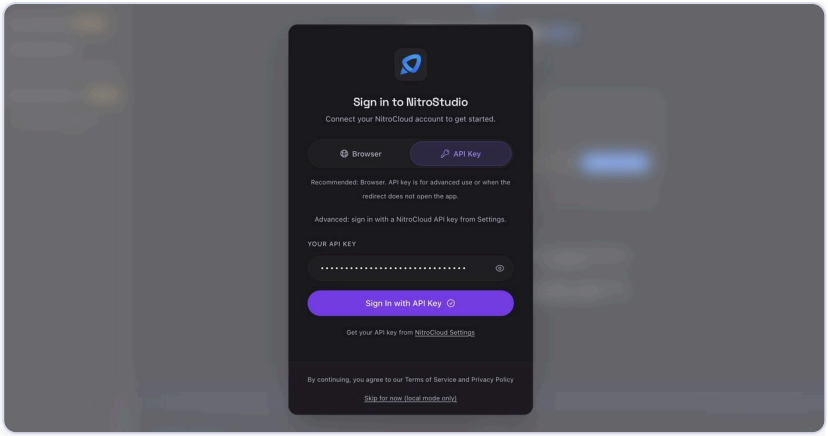
The API Key tab in the sign-in modal

2. Get your key: click **NitroCloud Settings** in the modal (opens `https://nitrocloud.ai/home/api-keys`) and copy an API key. Keys start with `nsk_live_`.



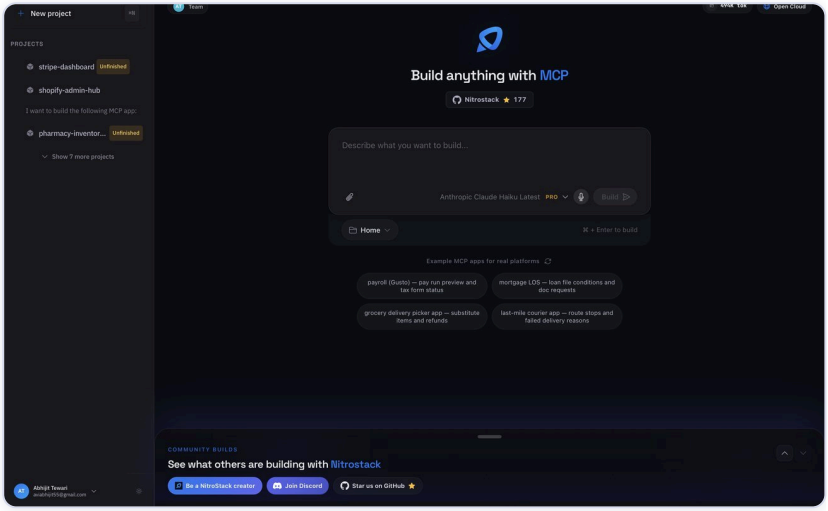
NitroCloud Settings — API keys page

3. Paste the key into the **Your API Key** field (placeholder `nsk_live_...`).



Pasting the API key into the field

4. Click **Sign In with API Key**. The button shows **"Validating..."** while it checks, then the modal closes on success.



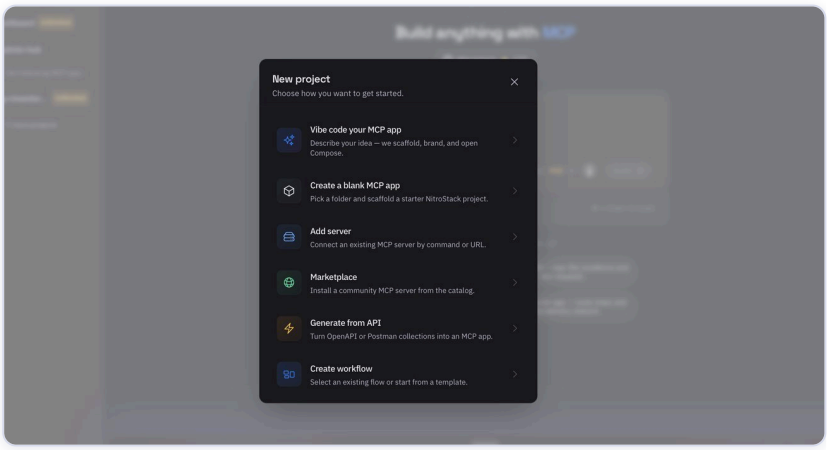
The Sign In with API Key button

> **Valid key format:** `nsk_live_` followed by 64 characters. Common errors: "Please enter an API key", "Invalid key format...", "Invalid, revoked, or expired API key".

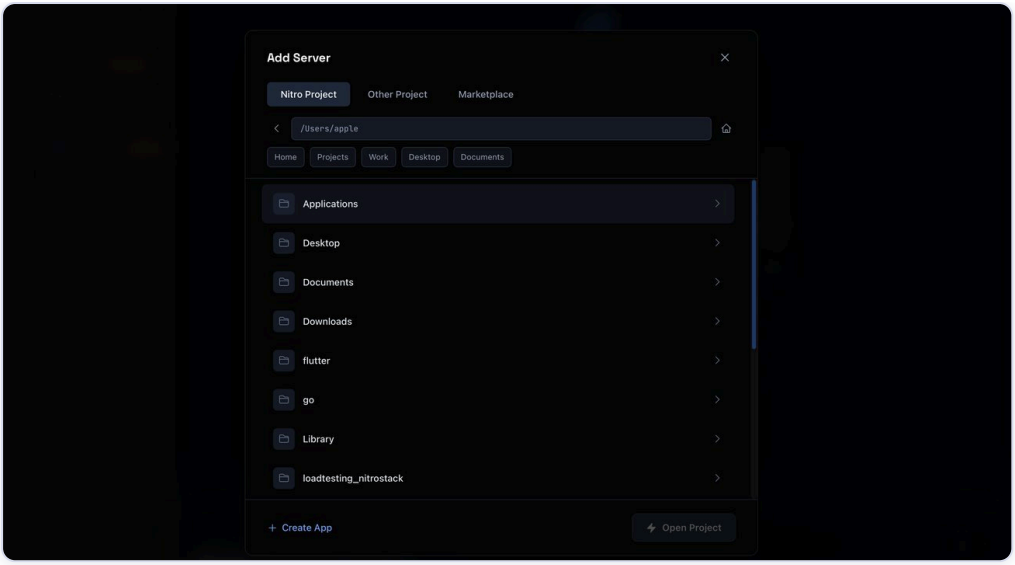
6. Studio Project Connect

Once you have an MCP server (a NitroStack project on disk, a custom STDIO command, or a running HTTP server), connect it in Studio via the **Add Server** modal.

Open it from the launcher (**Add MCP Server** / **New project**) or, inside Studio, from the sidebar → **Add / Manage Projects** → **Add New Project**. The modal is titled **"Add Server"** with tabs: **Nitro Project**, **Other Project**, **Marketplace**.



The Add Server modal



Add Server modal — tabs and options

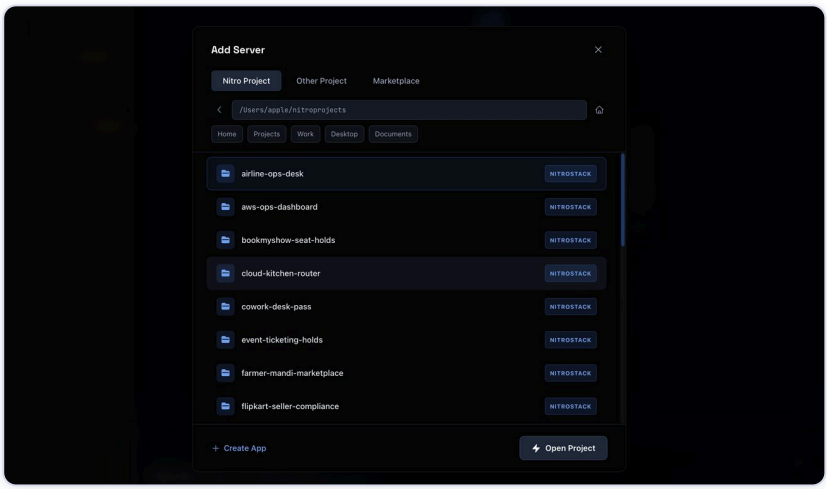
> **Prerequisite:** Both STDIO and HTTP connections require the **desktop app** (STDIO spawns a process; HTTP needs CORS bypass).

Connect via STDIO

STDIO means Studio spawns/talks to a local MCP process over standard input/output. There are two ways.

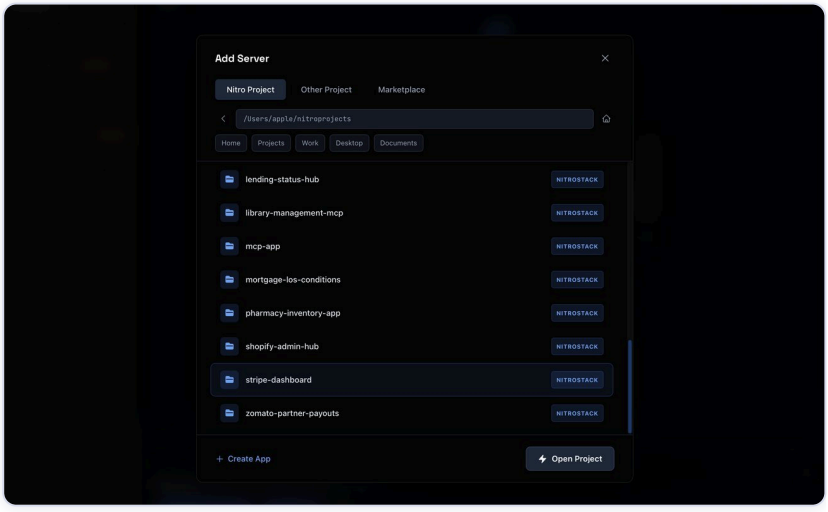
Option A — Nitro Project (recommended for NitroStack projects on disk)

1. Open **Add Server** → select the **Nitro Project** tab.



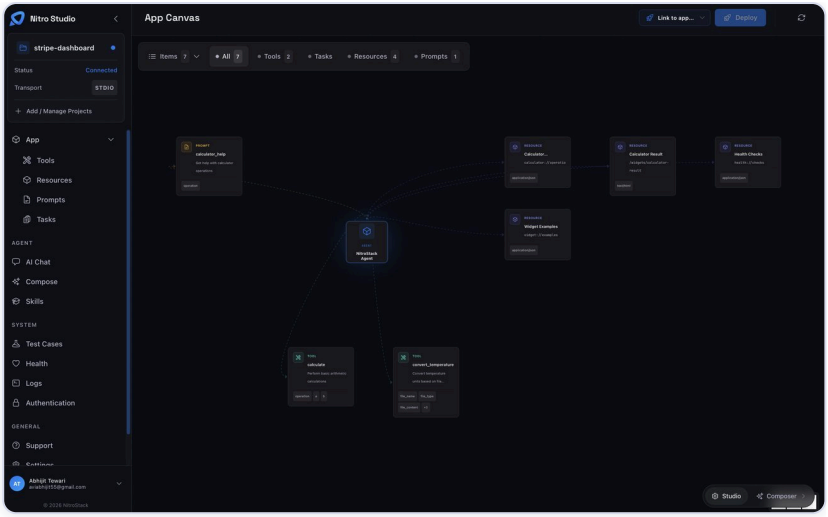
Add Server modal, Nitro Project tab

2. Browse to your project folder (quick chips: Home, Projects, Work, Desktop, Documents). Folders that are valid projects show a **NitroStack** badge.



Folder browser with a NitroStack-badged folder

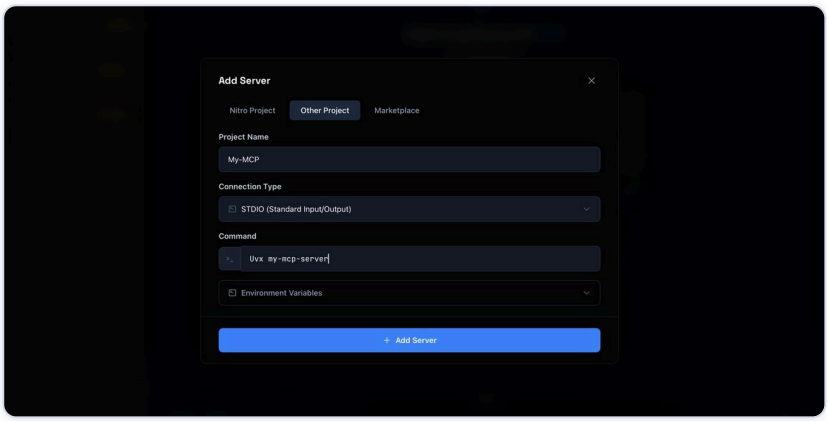
3. Click **Open Project**. For a NitroStack project you'll then choose how to open it — **Studio App Canvas** (explore tools, resources, widgets) or **Vibe Code (Compose)**. Choose **Studio App Canvas** to inspect. Studio runs `npx tsx src/index.ts` for you (no need to run `npm run dev` yourself; it auto-runs `npm install` if dependencies are missing).



The "Open Project" choice: Studio App Canvas vs Vibe Code

Option B — Other Project → STDIO (custom command)

1. Open **Add Server** → **Other Project** tab and enter a **Project Name** (e.g. "My Custom Server").
2. Set **Connection Type** to **STDIO (Standard Input/Output)**.
3. Enter the **Command** (e.g. `uvx mcp-server-name`). Optionally add **Environment Variables** (KEY / value), then click **Add Server**.

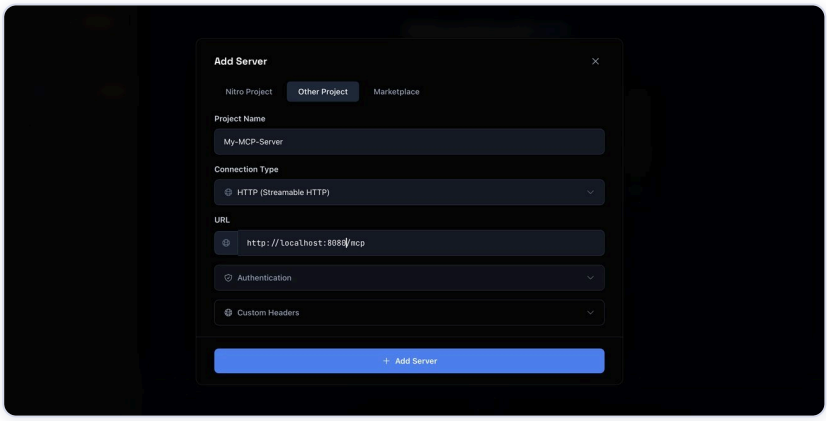


Other Project tab configured for STDIO with a command

Connect via HTTP

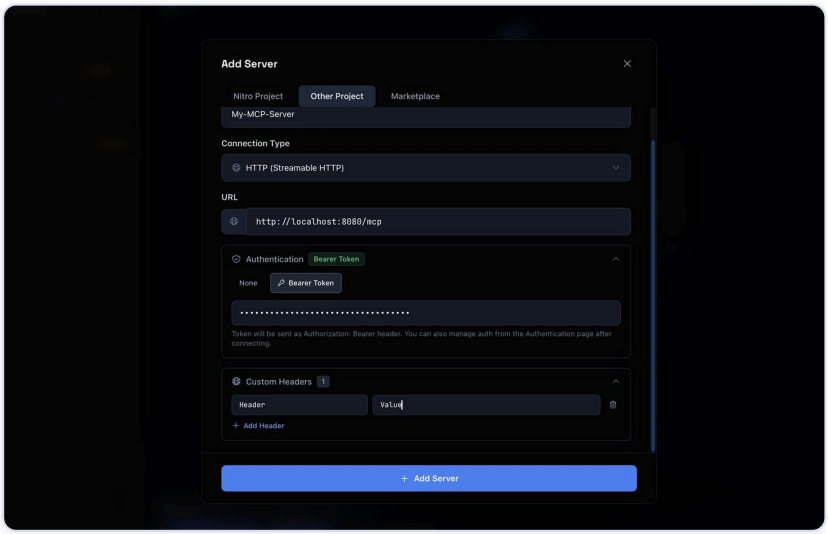
Use this to connect to an already-running remote MCP server.

1. Open **Add Server** → **Other Project** tab and enter a **Project Name**.
2. Set **Connection Type** to **HTTP (Streamable HTTP)** and enter the server **URL** (placeholder `http://localhost:8080/mcp`).



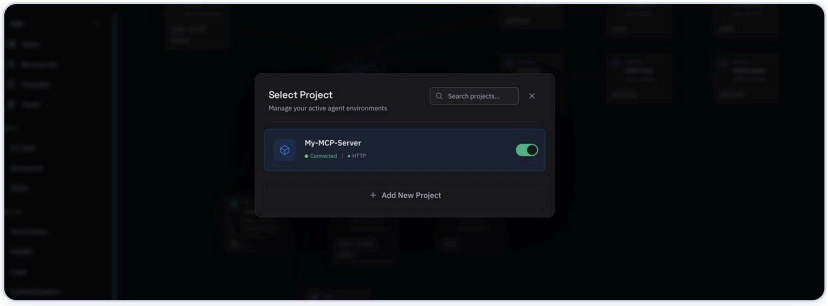
Other Project tab configured for HTTP with a URL

3. (Optional) Under **Authentication**, choose **Bearer Token** and paste a token (sent as `Authorization: Bearer`). You can also add **Custom Headers** (Header-Name / value).



HTTP authentication — Bearer token and custom headers

4. Click **Add Server**. After connecting, the sidebar shows an **HTTP** transport badge.



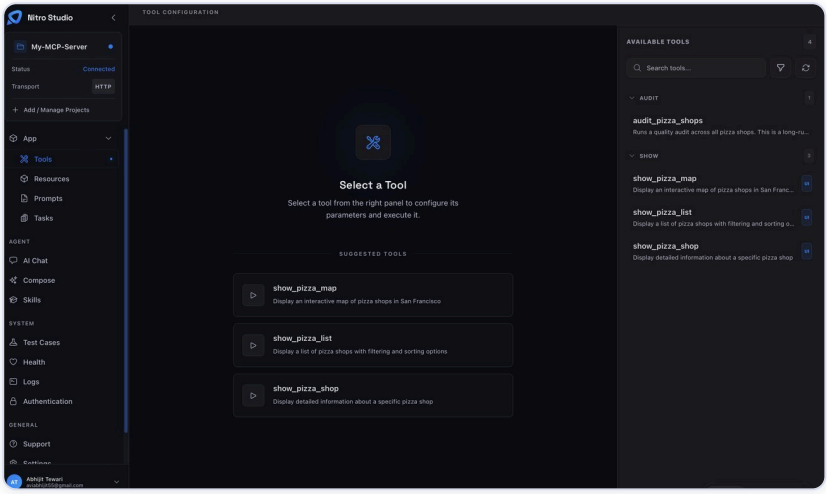
Connected HTTP project with the HTTP badge

7. Studio Features

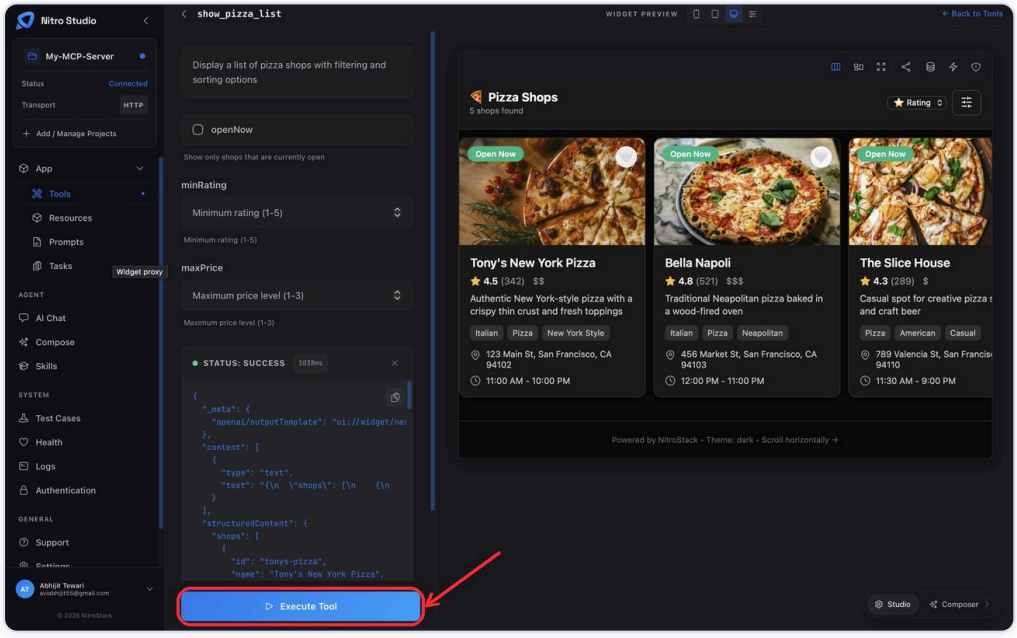
Once connected, use these Studio pages. Navigation is in the left sidebar, grouped under **App**, **AGENT**, and **SYSTEM**.

Tools

1. In the sidebar, under **App**, click **Tools** (route `/tools`).
2. The right panel lists **Available Tools** (with a count and search). Click a tool, fill in its inputs (generated from its `inputSchema`), then click **Execute Tool**. The JSON result appears with a **Status: Success** indicator. Tools with a widget show a **Widget Preview** (Mobile/Tablet/Desktop viewports).
3. Optionally use **Run as Task** for async execution.



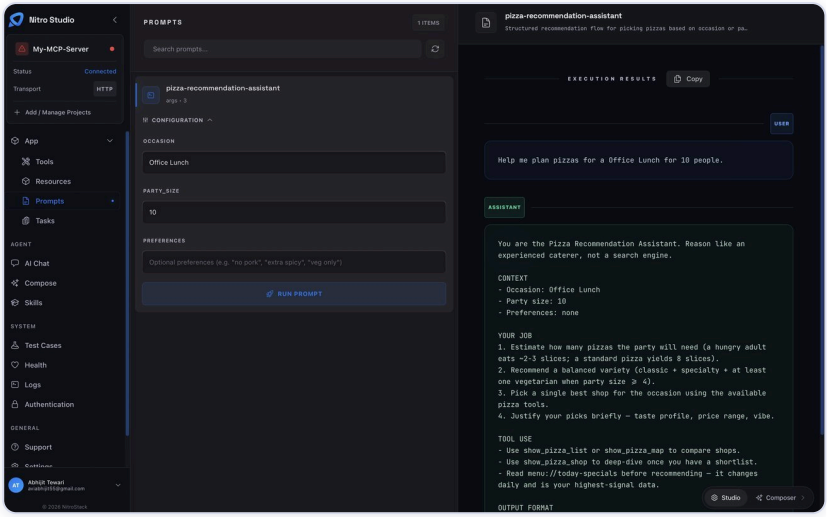
Tools page — list, inputs and Execute Tool



Tools page — result and widget preview

Prompts

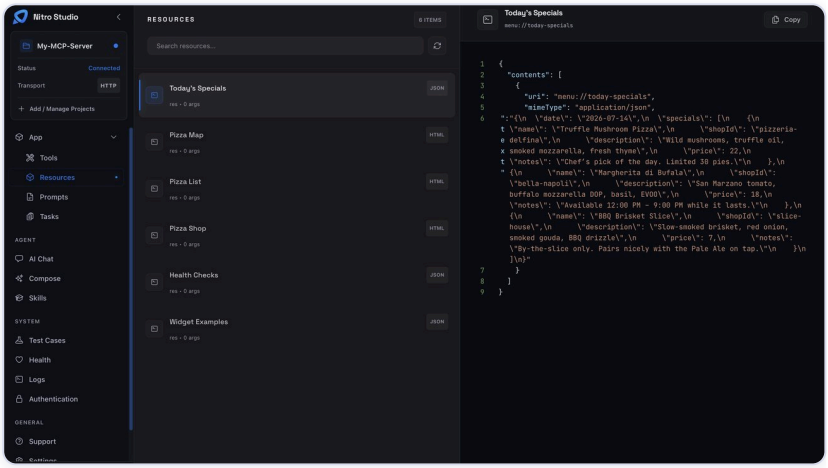
1. In the sidebar, under **App**, click **Prompts** (route `/prompts`).
2. Select a prompt. If it takes arguments, expand **Configuration**, fill fields, then click **Run Prompt**.
3. Results appear on the right under **Execution Results** (role-labeled messages). Use **Copy** to copy the output.



Prompts page — configuration and results

Resources

1. In the sidebar, under **App**, click **Resources** (route `/resources`).
2. Select a resource (type badges: JSON, CSV, YAML, etc.). If its URI has parameters (`{param}`), expand **Arguments Configuration**, fill them, and click **Fetch Resource**.
3. Content shows on the right (syntax-highlighted); widget resources show a preview with **Select Preview Case** examples. Use **Copy** to copy content.

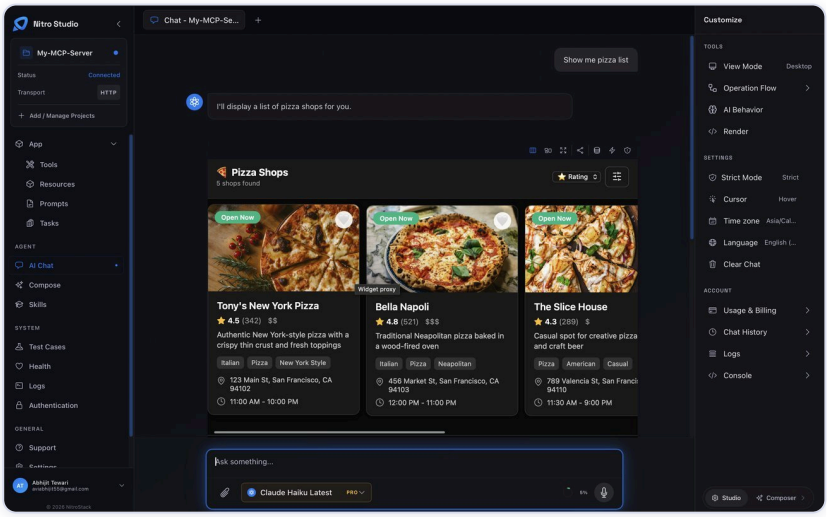


Resources page — content and preview

AI Chat

> Requires being signed in to NitroCloud (see Studio Login). Without it you'll see a "Sign in to use AI Chat" wall.

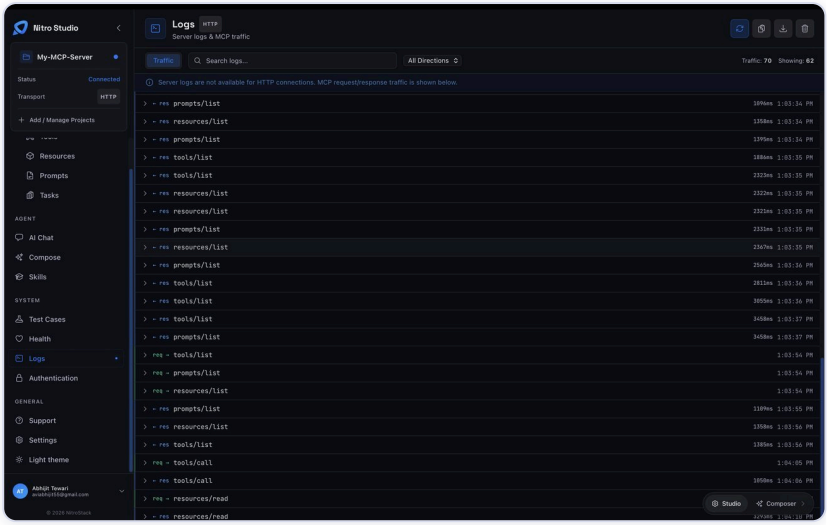
1. In the sidebar, under **AGENT**, click **AI Chat** (route `/chat`). Type in the input (placeholder "Ask something...") and send. The model can call your MCP tools automatically.
2. When the AI wants to call a tool, a **Tool approval** modal appears — click **Allow** or **Deny**. Pick a model with the model picker, attach files, or open **Customize** for view mode, usage/billing, chat history, and logs.



AI Chat page — conversation with a tool call

Logs

1. In the sidebar, under **SYSTEM**, click **Logs** (route `/logs`). Subtitle: "Server logs & MCP traffic".
2. Use the tabs **All**, **Server Logs**, **Traffic** to filter (HTTP-only projects show **Traffic** only). Filter with **Search logs...**, **All Levels** (info/warn/error/debug), and **All Directions** (Requests/Responses).
3. Pause/resume streaming, toggle auto-scroll, copy all, download JSON, or clear. Expand a row to see `params` / `result` / `error` payloads.



Logs page — tabs, filters and traffic

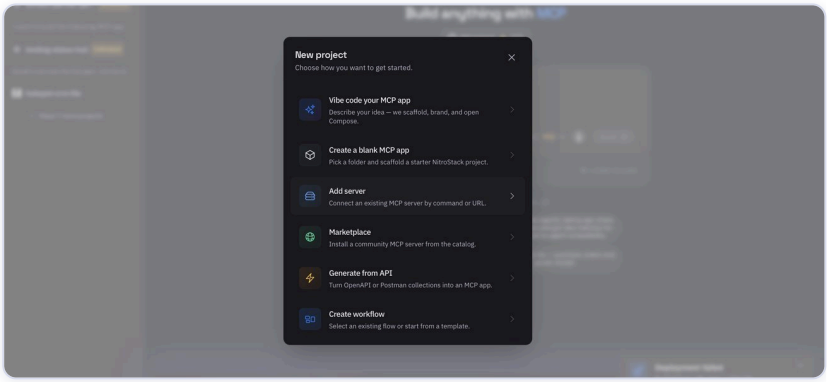
8. Vibe Coding

"Vibe Coding" is the **Compose** workspace — an AI coding agent that scaffolds and edits your project, runs the dev server, and surfaces tools in a live preview. Then you push it to the cloud and connect it to ChatGPT.

> **Prerequisites:** Signed in to NitroCloud, using the **desktop app**, and a project connected. Compose uses NitroCloud credits to run the agent.

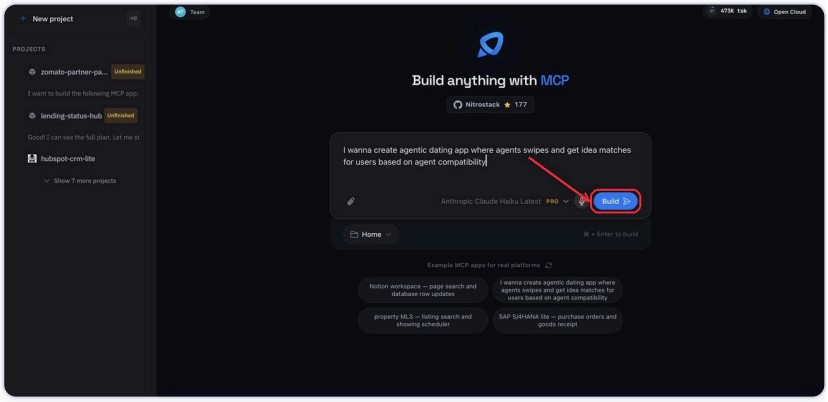
How to do Vibe Coding — the entire flow

1. **Enter Compose.** Start from any of: Launcher **New project** → **Vibe code your MCP app**, the **Open Project** modal → **Vibe Code (Compose)**, the sidebar (**AGENT**) → **Compose**, or the bottom-right workspace switcher → **Composer**.



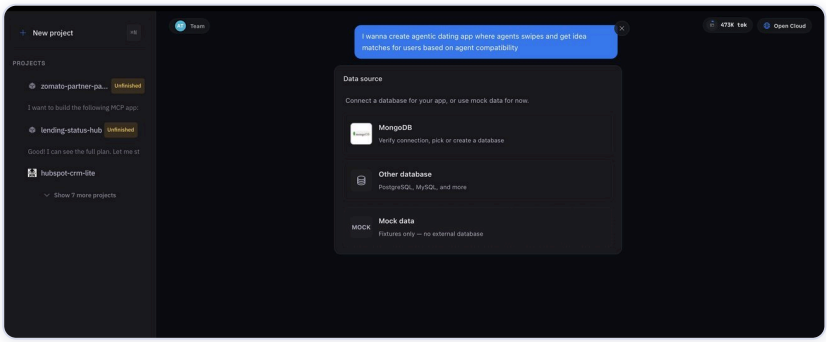
Entering Compose / Vibe Coding

2. **Describe your app and pick an agent.** On the "Build anything with MCP" screen, choose one of the example topics or type your own prompt / short description of the MCP you want to build. Pick the model in the **agent dropdown** (e.g. Anthropic Claude Haiku Latest), then click **Build**.



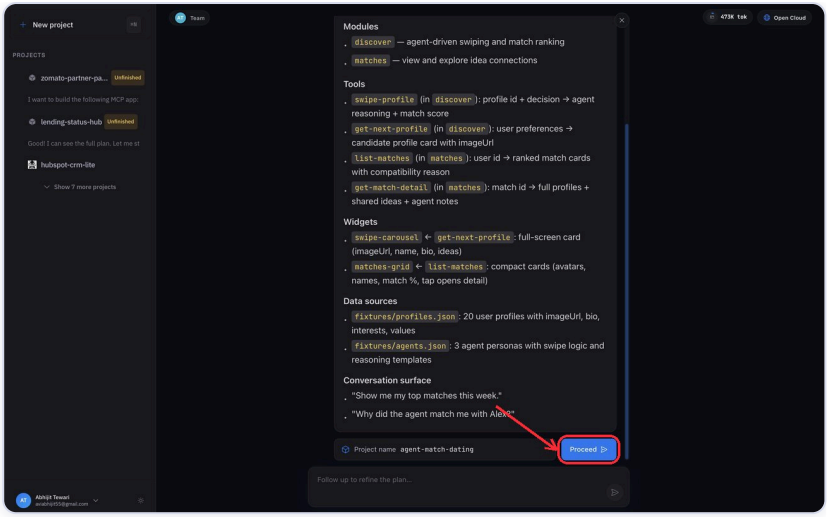
Describe your app and choose the agent

3. **Choose your data source.** Select **MongoDB**, **Other database** (PostgreSQL, MySQL, and more), or **Mock data** (fixtures only — no external database).



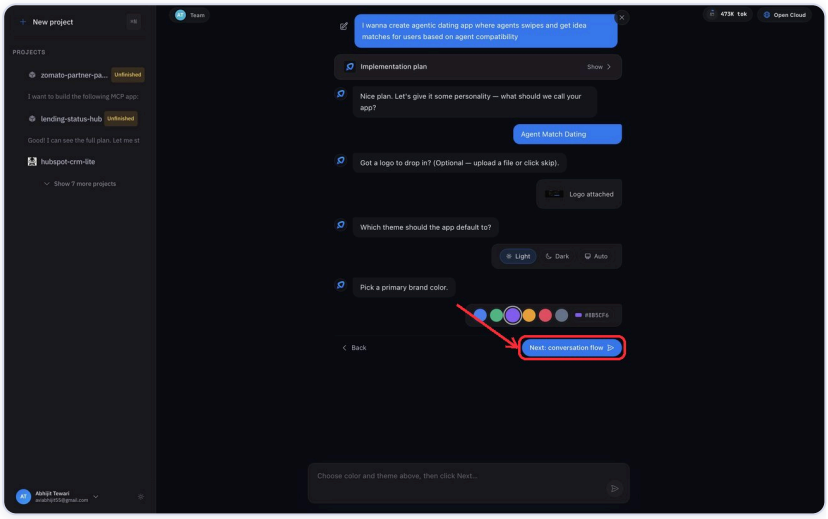
Choosing the data source

4. **Review the build plan.** Compose shows the plan for your MCP — modules, tools, widgets, data sources and the conversation surface. Give it a project name and click **Proceed**.



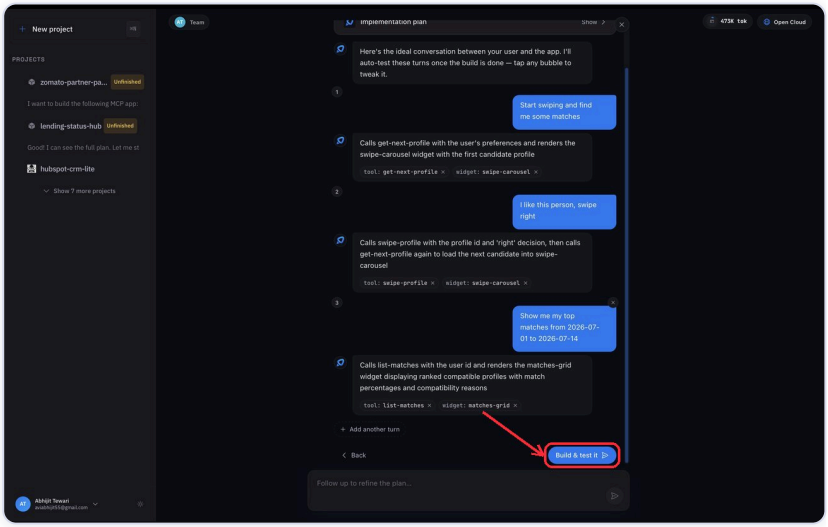
The generated build plan — click Proceed

5. **Customize your chat.** Set the app **name**, optionally **attach a logo**, pick the **theme** (Light / Dark / Auto) and a **primary brand colour**, then click **Next: conversation flow**.



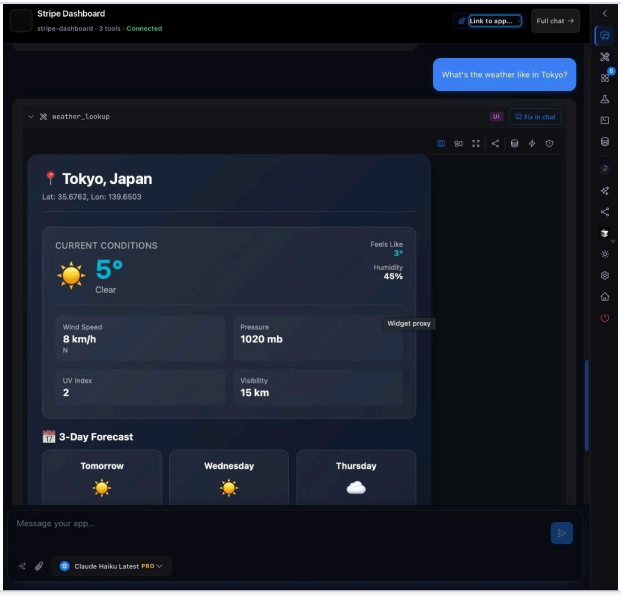
Customizing name, logo, theme and brand colour

6. **Review tools & widgets.** Compose shows the tools and widgets it's going to create and an example conversation. Add more via **Add another turn**, or click **Build & test it** to start building.



Tools and widgets to be created — Build & test it

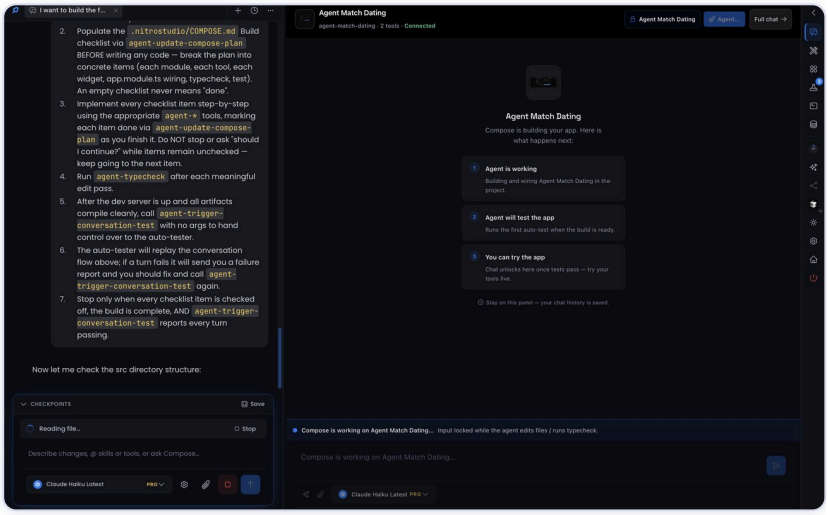
7. **Studio connects the project.** A loader shows **MCP Server:** and **Widget Dev Server:** status (Connecting… → Connected / Ready). If it fails, click **Retry Connection**. Once connected, the header shows **Connected** and the right-rail MCP chat renders your widgets live.



Project connected — live MCP chat rendering a widget

Prompt the agent

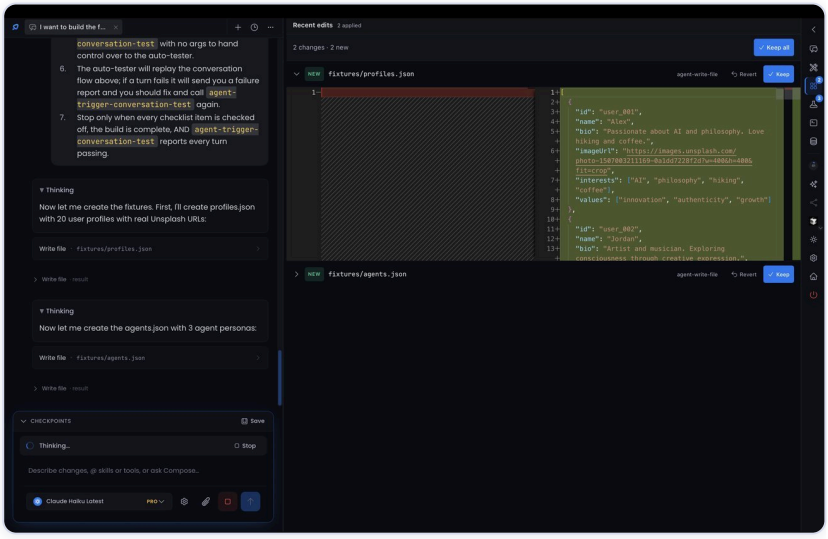
In the left chat pane, describe what you want to build (placeholder "Describe changes, @ skills or tools, or ask Compose…"). Suggestion chips are available on the empty state.



Prompting the Compose agent

Watch it work

The agent streams its work: a status bar shows **Thinking… / Working… / Reading file… / Running typecheck…**, plan checklists appear, and file writes stream in as cards. If the agent wants to run a shell/terminal command, a **"Compose wants to run a tool"** modal asks you to **Allow** or **Deny** (in Ask mode). File edits apply immediately.



The agent working — streaming status and edits

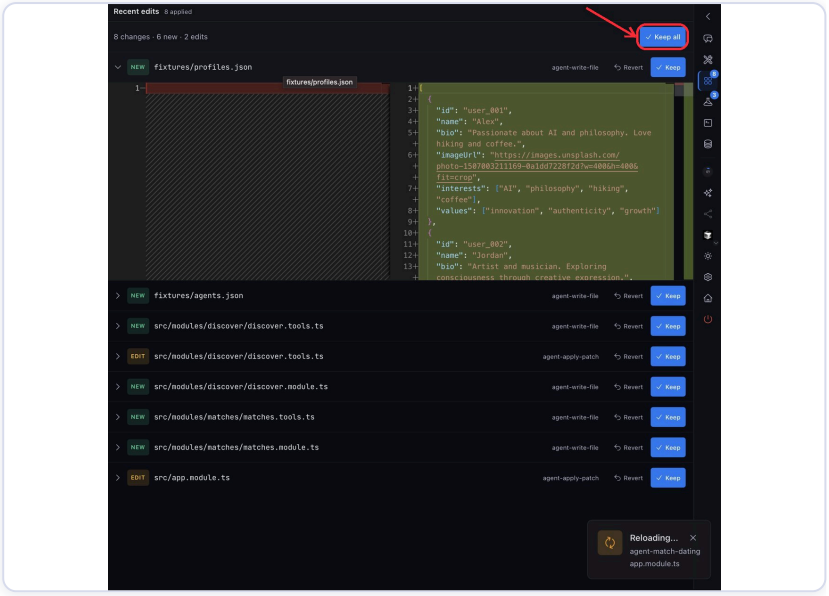
Try it live

The right rail's **MCP chat** panel is a live preview: chat with your MCP server's tools and see widgets render as the agent builds. The dev server hot-reloads. (See the connected live-preview screenshot under step 7 above.)

How to check changes

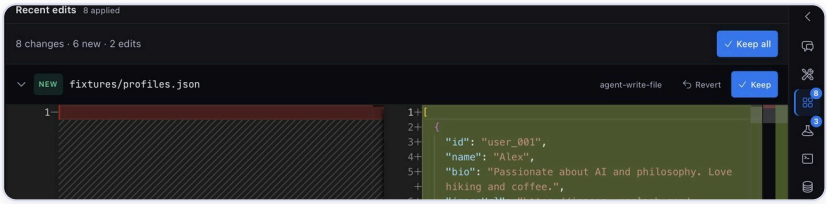
There's no button literally called "Check changes" — review agent edits in the right rail.

1. Open the **Diff queue** panel (right-rail icon; it auto-opens when the first edit lands). Header: **Recent edits**. Each changed file shows a **NEW** or **EDIT** badge and a side-by-side diff. Per file, click **Revert** (restore previous contents) or **Keep** (acknowledge). Use **Keep all** to clear the queue.



Diff queue — Recent edits with Keep / Revert

2. For broader undo, open **Checkpoints** in the bottom chat dock. Click **Save** to snapshot the working tree; click the revert (↩) icon on any checkpoint to restore to that point.



Checkpoints — Save and revert

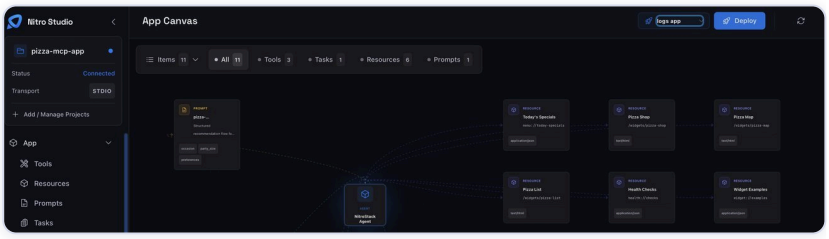
9. Cloud & Deployment

Ship your MCP app to **NitroCloud**, then deploy from Studio, NitroCloud, or GitHub.

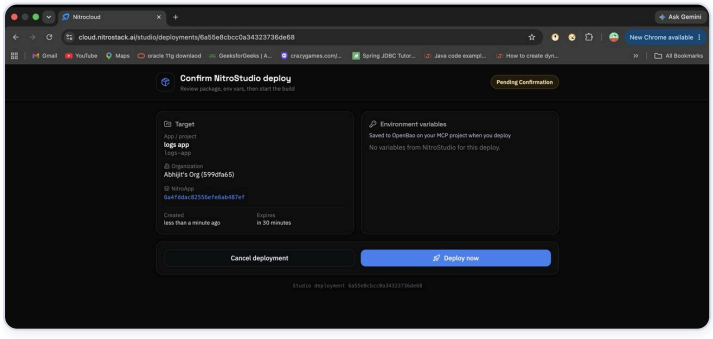
Deploy from Studio (App Canvas / Compose)

Deployment is not a sidebar page — it lives on the **App Canvas** (`/`) and inside **Compose**, shown via a modal + status toast.

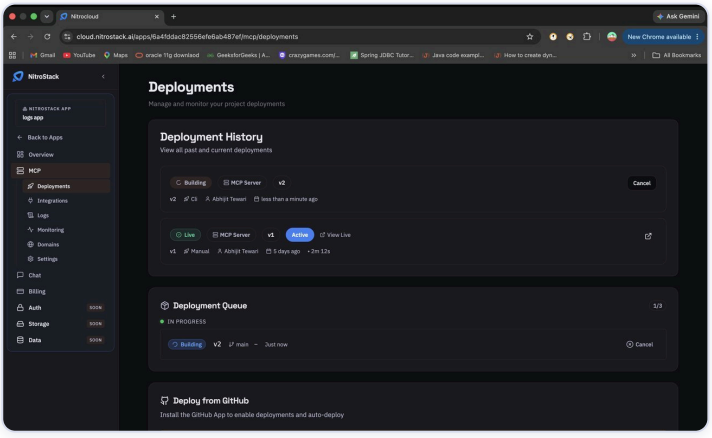
1. On the **App Canvas** header (or the Compose MCP chat header), if no cloud app is linked, use **Link to app…** to pick one, or **Create Cloud App**.
2. Click **Deploy**. The **Deploy to NitroCloud** modal opens and walks through: **Preparing bundle → Uploading project → Waiting for confirmation → Building and deploying → Deployment live**.
3. At the confirmation step, click **Open Confirmation Page** (confirms in your browser). You can also **Run in background**. When live, the modal shows the service URL with **Copy** and **View deployments in NitroCloud**.



Deploy to NitroCloud modal — steps



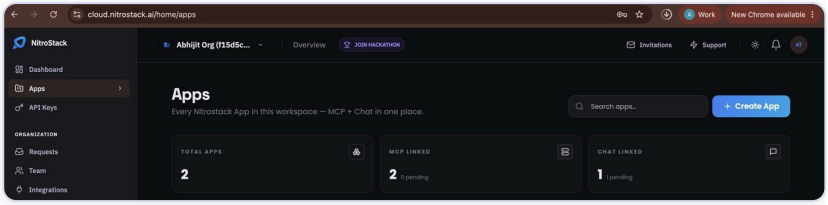
Deploy to NitroCloud modal — building and deploying



Deploy to NitroCloud modal — deployment live

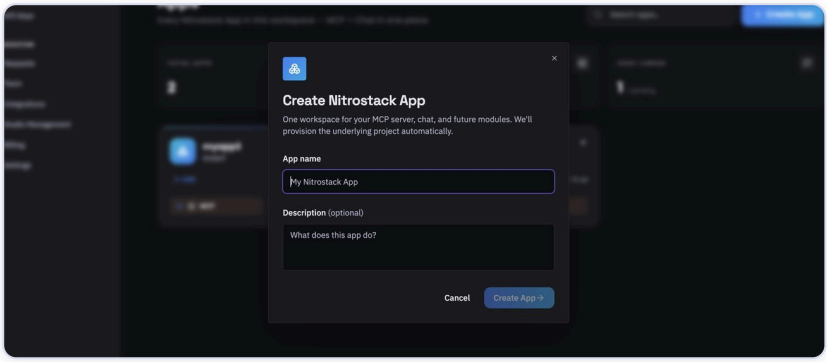
Create a Project in the cloud

1. Go to NitroCloud (`https://nitrocloud.ai`). On **/home** or **/home/apps**, click **Create Nitrostack App** (also surfaced as **Create New App** / **Create App**).



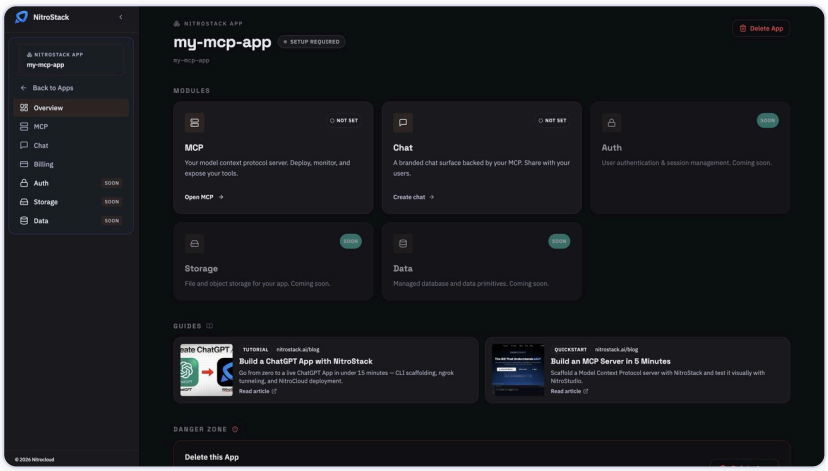
NitroCloud — Create Nitrostack App button

2. In the **Create Nitrostack App** modal, enter an **App name** (min 2 chars) and an optional **Description**. Click **Create App**.



Create Nitrostack App modal

3. On success you're taken to the app at `/apps/:id`, with a sidebar for **Overview** and **MCP** (Deployments, Integrations, Logs, Monitoring, Domains, Settings).



New app Overview page

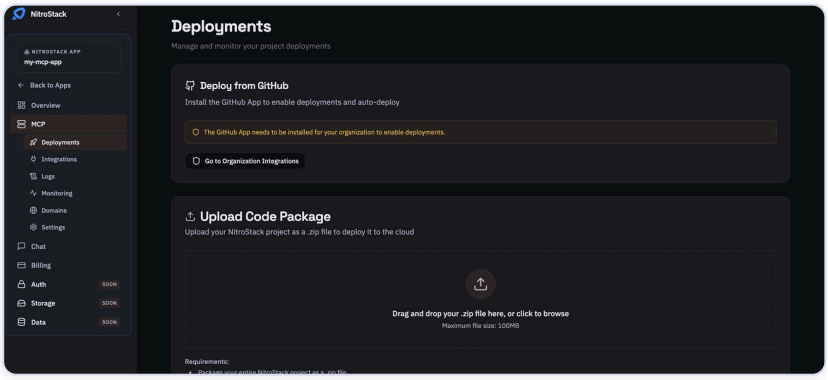
Create a Deployment

From Compose (Studio):

1. In the Compose MCP chat header, use **Link to app…** to select the cloud app you created (or **Create App**).
2. Click **Deploy**. Compose first runs `npm run build` locally; if it fails, the error goes back to the agent to fix. On success, the **Deploy to NitroCloud** modal opens and uploads/deploys.

Compose deploy controls — Link to app and Deploy

From NitroCloud directly: on `/apps/:id/mcp` ("Ship your MCP server") pick a path — **Start from CLI** (Recommended): `npm i -g @nitrostack/cli` then `nitrostack-cli init <app-slug>` ; **Connect GitHub** → **Connect repository**; or **Upload a code package** → drag a `.zip` (max 100MB) → **Upload & Deploy**.

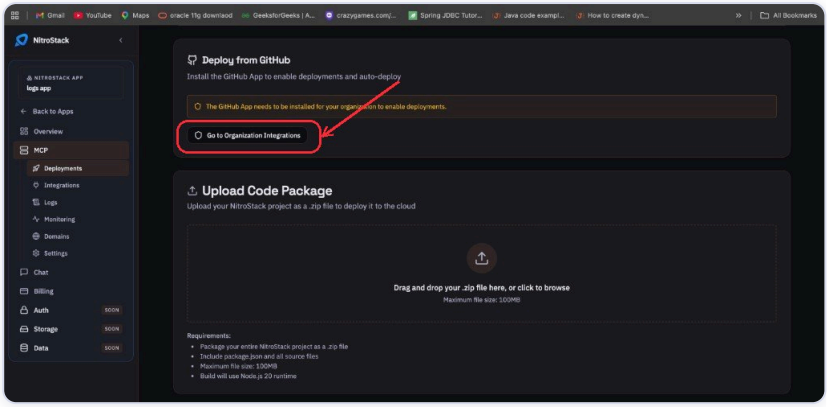


NitroCloud "Ship your MCP server" — deploy paths

Deploy from GitHub (auto-deploy)

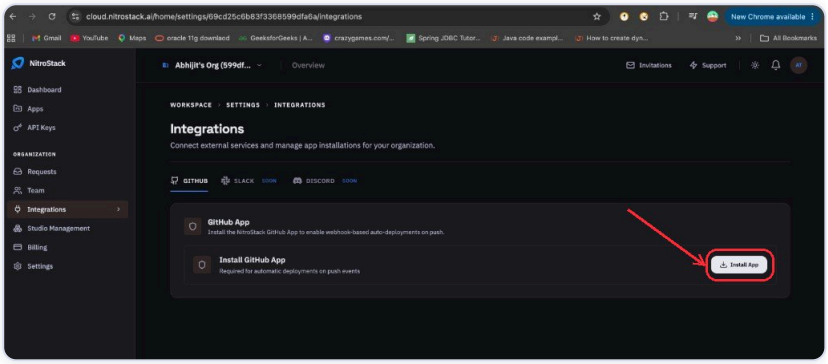
Connect a repository once, then every push to the linked branch deploys automatically.

1. Open the app's **Deployments** page (**MCP → Deployments**). If GitHub isn't connected yet, the **Deploy from GitHub** card reads *"The GitHub App needs to be installed…"* — click **Go to Organization Integrations**.



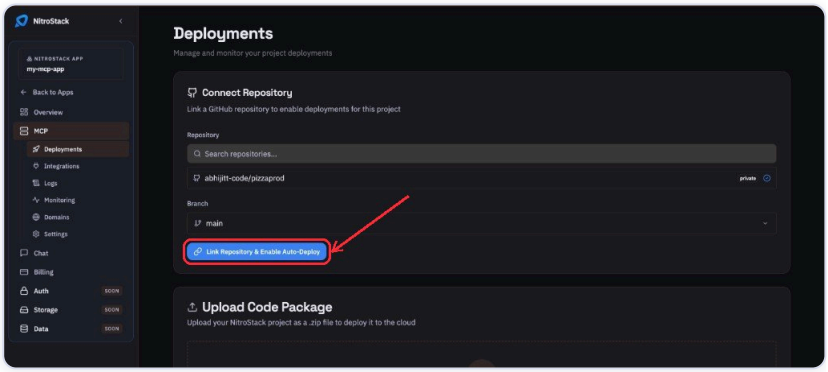
Deployments — Deploy from GitHub before the GitHub App is installed

2. On the **Integrations** page, under **GitHub App**, click **Install App** and authorise NitroStack on GitHub for your organization.



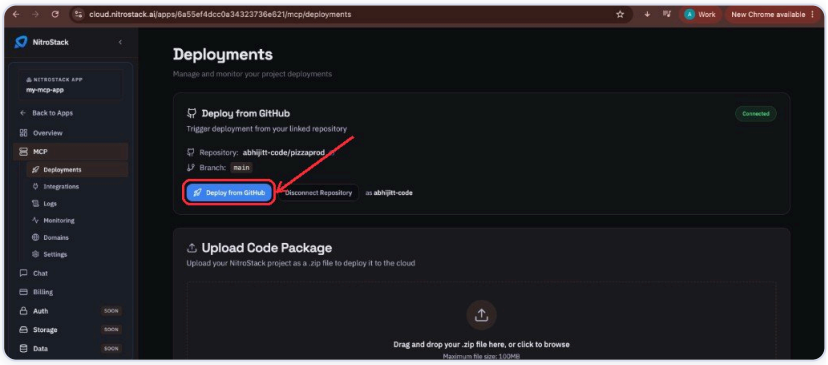
Organization Integrations — install the NitroStack GitHub App

3. Back on **Deployments → Connect Repository**, search and select your repository, choose the **branch** (e.g. `main`), then click **Link Repository & Enable Auto-Deploy**.



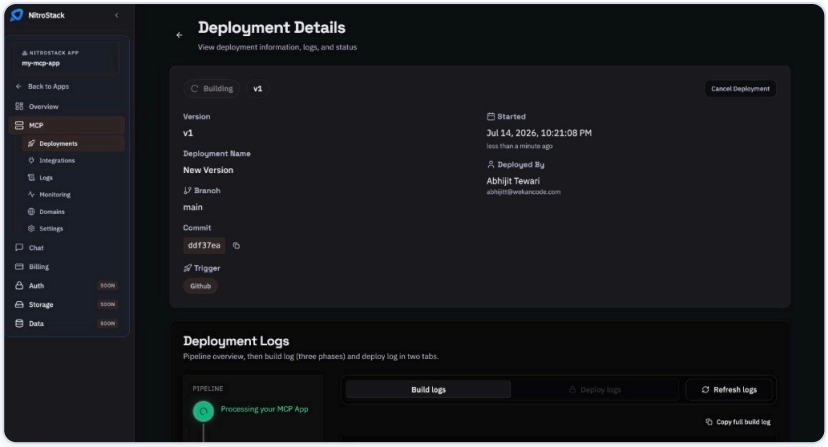
Connect Repository — link a repo and enable auto-deploy

4. The card now shows **Connected**. Click **Deploy from GitHub** to trigger a deployment (it also auto-deploys on every push to the linked branch).



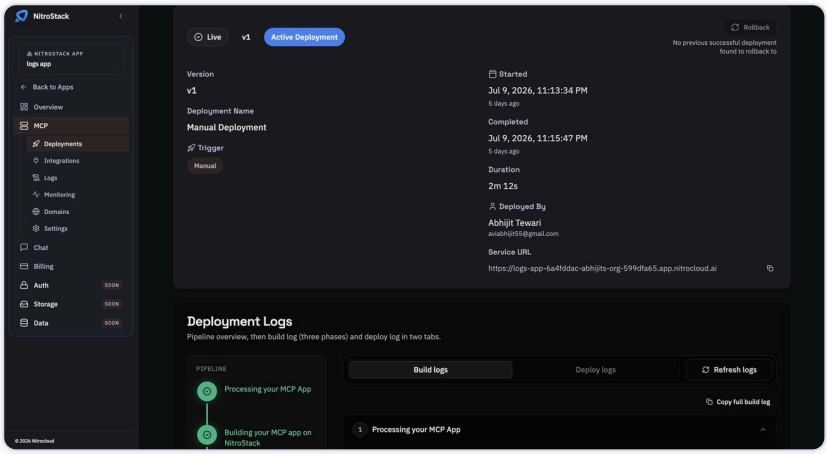
Deploy from GitHub — repository connected, ready to deploy

5. The **Deployment Details** page opens and streams the pipeline (**Processing** → **Building** → **Deploying**) with build & deploy logs. When it turns **Live**, the **Service URL** appears.



Deployment Details — building and deploying the MCP app

Watch the deployment progress. Whichever path you pick, the status moves through **Pending** → **Building** → **Deploying** → **Live**. When **Live**, open the deployment details to find the **Service URL** (with **Copy** / **Open**).

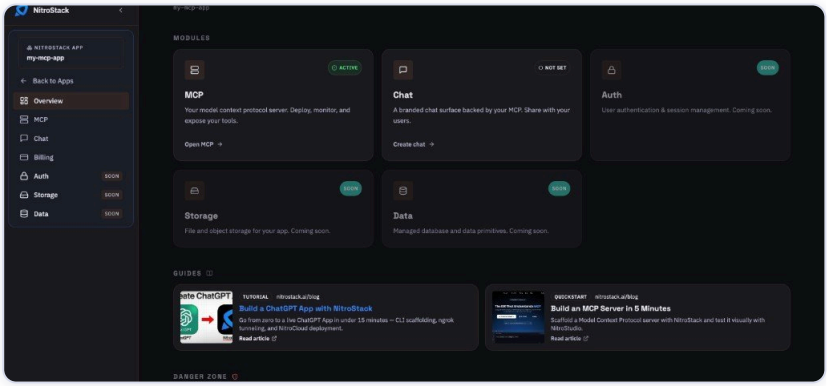


Deployment details — status Live and Service URL

10. Connect to ChatGPT

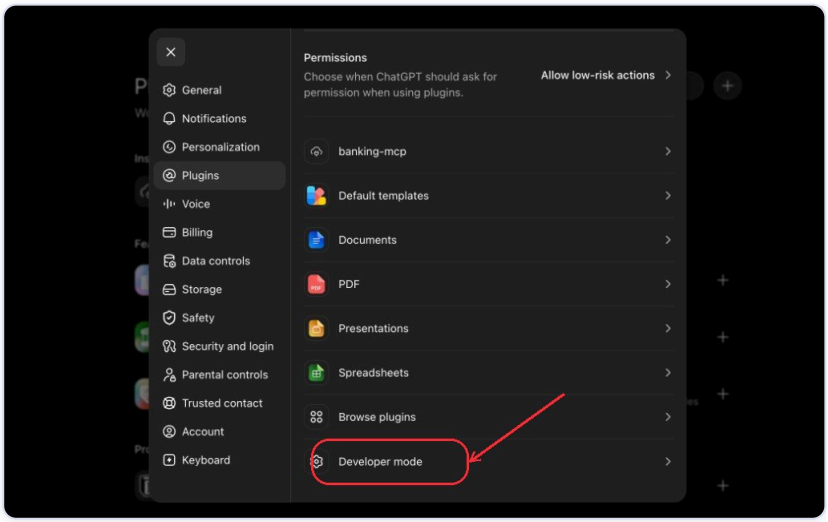
Once your MCP server is **Live**, connect it to ChatGPT via NitroCloud's guide.

1. On the live MCP app **Overview**, open the **MCP** module (or the **"Build a ChatGPT App with NitroStack"** guide) to find the **Deploy to ChatGPT** step and your **MCP URL**.



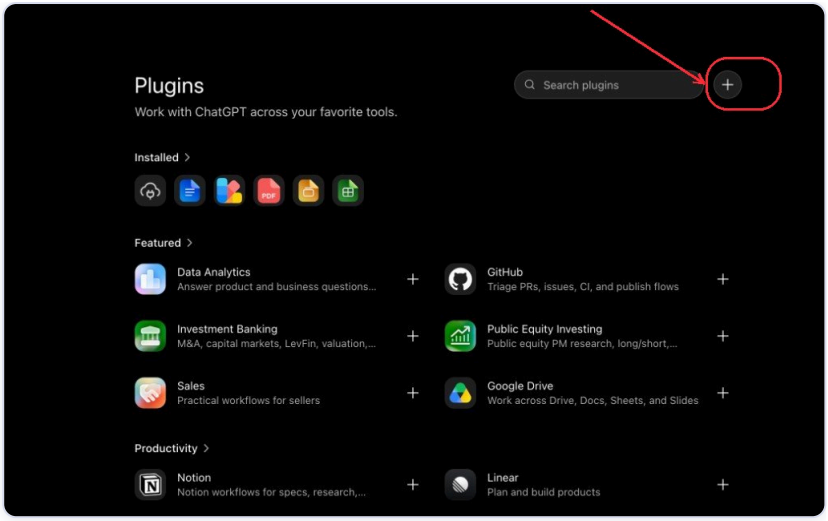
NitroCloud app Overview — MCP module and the ChatGPT guide

2. **Enable Developer Mode.** In ChatGPT, open **Settings** → **Plugins** (Apps) and select **Developer mode**. (Requires ChatGPT Plus or Pro.)



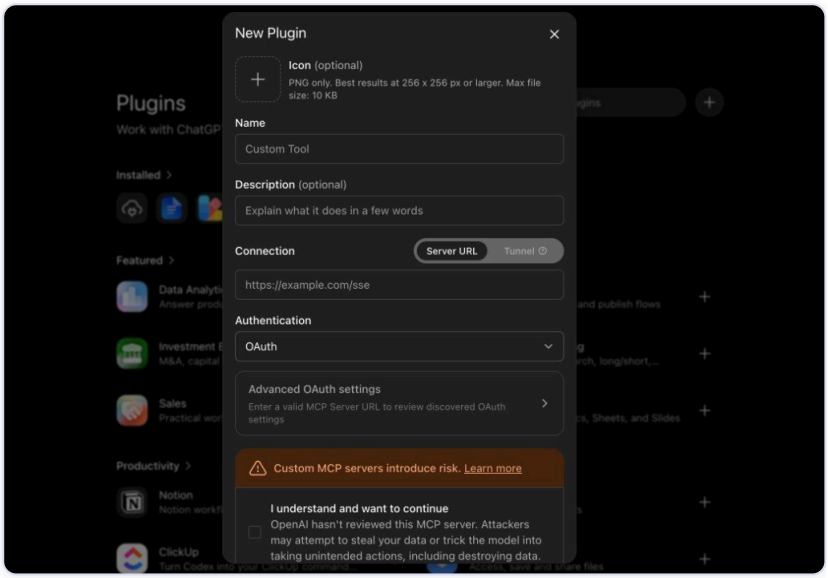
ChatGPT Settings → Plugins → Developer mode

3. **Open the plugins page** and click the **+** (add) button to start creating a new app.



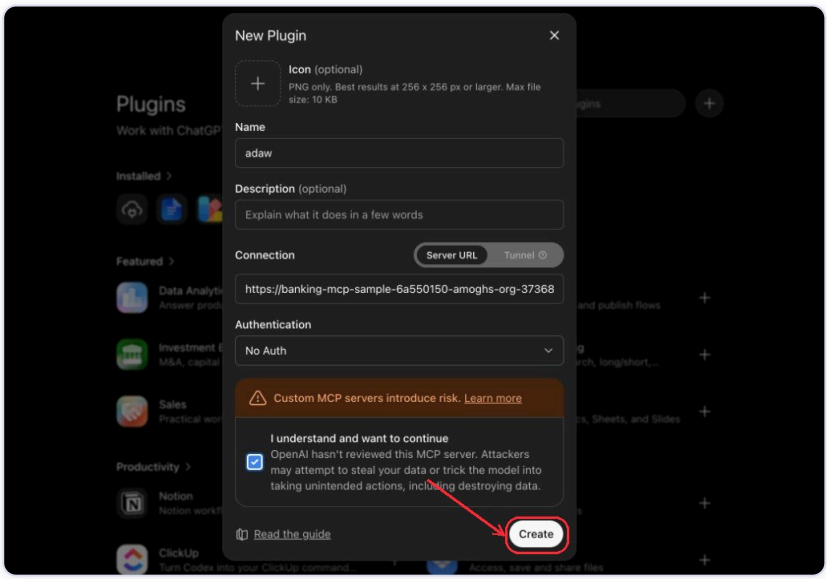
ChatGPT Plugins page — the + button to add a plugin

4. **Set up the connection.** In the **New Plugin** dialog, keep **Connection** on **Server URL** and pick the **Authentication** — **No Auth** for open servers, or **OAuth** for protected ones.



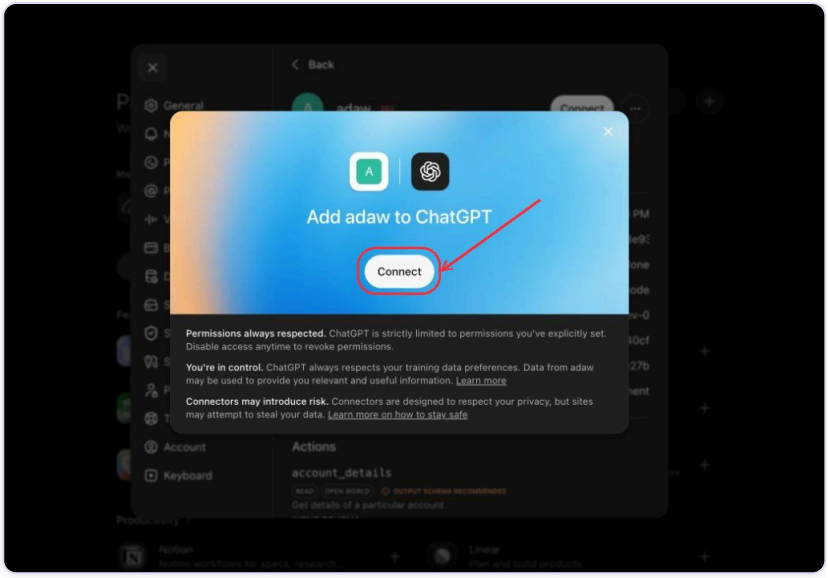
New Plugin dialog — connection type and authentication

5. **Paste your MCP URL and create.** Enter a **Name**, paste your **MCP URL** (`{serviceUrl}/sse`) into the **Server URL** field, tick **"I understand and want to continue"**, then click **Create**.



New Plugin — name, MCP URL and the Create button

6. **Connect the app.** On the **"Add {app} to ChatGPT"** screen, click **Connect**. ChatGPT loads your tools.



Add your app to ChatGPT — Connect

7. **Test it.** Open a new chat and ask something that triggers your app's tools. ChatGPT invokes them through your MCP server and renders the response (including widgets).

> **Where to find the MCP URL:** It's `{serviceUrl}/sse` — shown in the **Deploy to ChatGPT** modal (field **MCP URL**, with a copy button). The base **Service URL** is on the deployment details page.

11. Troubleshooting

How to Restore

Studio has several restore paths depending on what you want to undo.

A. Compose — Checkpoints (whole project state)

In Compose, open the **Checkpoints** section in the bottom chat dock. Click **Save** to snapshot the current working tree, or click the revert (↩) icon on any listed checkpoint (`{sha} · {time}`) to **Revert to this checkpoint**. This uses a session shadow branch, so your main git HEAD is untouched.

B. Compose — Diff queue (per-file)

Open the **Diff queue** → **Recent edits**. Click **Revert** on a file to restore its previous contents, or **Keep** to accept it.

C. Workflows — Version History

In a workflow canvas, open the hamburger menu → **Version History**. Click **Restore** on a saved version. Studio auto-creates a backup before restoring and toasts "Restored to snapshot".

D. Chat — Restore a closed conversation

In AI Chat, open the history sidebar → **Closed / Cleared** section. Click the ↻ **Restore chat** button on an archived conversation.

Different troubleshooting steps

Problem	What you'll see	Fix
Project won't connect	"Connection Failed" + **Retry Connection**	Click **Retry Connection**; read the error text below it.
STDIO connection fails	Project won't connect over STDIO (Nitro Project / Other → STDIO)	Switch to **HTTP**: open **Add Server** → **Other Project** tab → set **Connection Type** to **HTTP (Streamable HTTP)** → paste your server URL (e.g. `http://localhost:3000/mcp` or whatever port your `npm run dev` prints) → **Add Server**. See Connect via HTTP.
Folder moved/deleted	"Project directory not found: '…'. The folder may have been moved or deleted."	Remove the project and add it again from its new location.
Node.js missing / not found	"Command '…' not found. Please ensure Node.js and npm are installed…"	Install Node.js 18+ (20.x recommended), or use **Install bundled Node.js** in onboarding.
Node.js too old	"Node.js {N} is too old" / "Upgrade to Node.js 18 or newer."	Upgrade Node from nodejs.org, then **Re-check**.
Dependencies not installed	"Dependencies not installed or out of date…" / "tsx is not available…"	Studio auto-runs `npm install` ; if it fails, run `npm install` manually in the project dir.
Login redirect stuck	"Waiting for login..." never completes	Wait ~10s, then **Switch to API Key** → **Sign In with API Key** with an `nsk_live_...` key.
Secure sign-in error	"Sign-in didn't complete securely. Close other NitroCloud login tabs…"	Close other NitroCloud login tabs, try once more, or use an API key.
Deployment failed	"Deployment failed" + reason (e.g. "Could not prepare the deployment package.")	Read the subtitle; if it's "Waiting for confirmation", confirm in your browser. A presigned upload URL expires in 15 min; a deployment expires in 30 min if not confirmed.
Pending deployment (400)	"You already have a pending deployment"	Cancel the existing pending deployment first, then redeploy.
Widget dev server timeout	"Widget dev server started on port {N} but did not respond within 45s"	Retry the connection; the agent can restart the dev server.
MCP unreachable after retries	"MCP server unreachable after 5 reconnect attempts — open the project to reconnect manually."	Reopen the project to reconnect.
Compose model error	"The model couldn't process this request. Retrying on Claude Haiku usually resolves it."	Click **Retry with Claude Haiku** (or **Retry last message**).
Widgets not loading	Widgets stay blank / don't render in the live MCP chat or preview	**Disconnect the MCP server and reconnect** to reload the widgets (in Compose click **Retry Connection**; on the App Canvas remove the project and add it again).